

SIMATS ENGINEERING

CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192311097
Name	DILLILOKESH D

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Code of Conduct:

*I Dillilokesh D 192311097 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

ANSWERS

1.

i. Greedy Best-First Search in this scenario

Greedy Best-First Search selects the next node only by the heuristic value $h(n)$, meaning it prefers the route that appears closest to the target. In a flood-affected region, this can make the drone move quickly toward the destination, which is useful when time is critical.

Strengths

- Very fast in many cases because it expands fewer nodes.
- Good for urgent decisions when a rough path is acceptable.
- Simple to implement and responsive in real-time environments.

Weaknesses

- It ignores the actual travel cost $g(n)$, so it may choose a path that looks short but is actually expensive or unsafe.
- It can get trapped in blocked or misleading regions because it follows the heuristic too aggressively.
- It does not guarantee the shortest or safest path under uncertainty.

ii. A* in the same situation

A* uses the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from the start and $h(n)$ is the heuristic estimate to the goal. This means the drone does not just chase the nearest-looking path; it also considers how costly the route has already become.

How it helps here

- If a road or path becomes blocked, A* can avoid paths that are cheap-looking but costly in reality.
- It is more reliable when terrain and weather cause variable movement costs.
- If the heuristic is admissible and consistent, A* can find an optimal path.

Limitation

- It usually takes more computation than Greedy Best-First Search, so it may respond slightly slower in highly dynamic conditions.

iii. Effect of heuristic accuracy

Heuristic accuracy strongly affects both algorithms, but in different ways.

- In Greedy Best-First Search, a good heuristic can make the search very fast, but a poor heuristic can mislead the drone into risky or dead-end routes.
- In A*, a better heuristic improves efficiency while still keeping the solution quality high.
- If the heuristic is too optimistic or inaccurate, A* may still work correctly, but it may explore more nodes and take longer.

So, heuristic quality is more critical for Greedy Best-First Search because it depends almost entirely on the heuristic, while A* is more robust because it also uses real path cost.

iv. Impact of dynamic blocked paths

Blocked paths create frequent changes in the search space. In such conditions:

- Greedy Best-First Search may repeatedly move toward a path that looks good but later becomes invalid, causing wasted time and possible rerouting.
- A* handles blocked paths better because it can evaluate alternate routes based on both actual cost and estimated distance.
- However, both algorithms may need to re-run if the environment changes after planning, which affects performance.

If updates are very frequent, repeated replanning becomes necessary, and the algorithm must be efficient enough to support that.

v. Recommended algorithm

A* is the more suitable algorithm for this scenario.

Justification

- It balances speed and path quality better than Greedy Best-First Search.

- It is more likely to choose a safer and lower-cost route in uncertain flood conditions.
- It provides better optimality, which is important when delivering emergency medicines.
- Although it may be slower than Greedy Best-First Search, the added reliability is worth it because safety and delivery success matter more than choosing the fastest-looking route.

2.

i. Optimization problem and why traditional search is inefficient

This traffic signal control problem can be formulated as an **optimization problem** where the AI system tries to find the best signal timings for hundreds of intersections to minimize:

- Total waiting time of vehicles.
- Fuel consumption caused by idling and stop-and-go movement.
- Traffic congestion across the city.

Formulation

- **State:** A particular set of traffic signal timings for all intersections.
- **Action:** Changing green, yellow, and red durations at one or more intersections.
- **Objective function:** A cost function combining waiting time, fuel use, and congestion.
- **Goal:** Find the signal timing configuration with the lowest total cost.

Why traditional search is inefficient

Traditional search methods like brute force, BFS, or DFS are not suitable because:

- The number of possible signal combinations is extremely large.
- The traffic environment changes continuously, so the search space changes over time.
- Evaluating every possible solution would take too long.

- The system needs real-time decisions, not exhaustive search.

So, traditional search becomes impractical due to **high dimensionality, dynamic conditions, and time constraints.**

ii. Hill Climbing in this scenario

Hill Climbing is a local search algorithm that starts with an initial traffic signal plan and repeatedly makes small changes that improve the objective function.

How it works

- Start with an initial signal timing setup.
- Evaluate nearby solutions by slightly changing timings.
- Move to the neighbor solution that gives the best improvement.
- Repeat until no better neighboring solution is found.

Example in traffic control

If one intersection has too much congestion, Hill Climbing may increase green time there and reduce red time, provided this improves the overall traffic flow.

Limitations

- It only looks at immediate neighbors, so it may miss better global solutions.
- It can get stuck in a poor local optimum.
- It may struggle when traffic patterns change suddenly.
- It does not explore widely across the solution space.

iii. Local maxima, plateaus, and ridges

These are common problems in local search and have real-world meanings in traffic optimization.

Local maxima

A local maximum is a solution that seems best compared to nearby options, but not the best overall.

- **Real-world meaning:** A signal timing plan may work well for one road section, but a better citywide coordination plan may exist elsewhere.
- Hill Climbing may stop here because no nearby change seems better.

Plateaus

A plateau is a flat region where many neighboring solutions have almost the same value.

- **Real-world meaning:** Changing a signal by a few seconds may not improve traffic flow noticeably.
- The algorithm may wander without finding progress.

Ridges

A ridge is a narrow path of improvement where the best solution is hard to reach by simple step-by-step movement.

- **Real-world meaning:** Improving traffic may require coordinated changes across multiple intersections, not just one signal at a time.
- Hill Climbing may fail because it moves in small local steps and cannot follow the narrow path effectively.

iv. Simulated Annealing and Genetic Algorithms

Both methods are better than simple Hill Climbing because they can escape local optima and explore the search space more effectively.

Simulated Annealing

Simulated Annealing sometimes accepts worse solutions early in the search.

- This helps it escape local maxima.
- Over time, the probability of accepting worse moves decreases.
- It is useful when traffic patterns are dynamic because it can adapt without being trapped too early.

How it helps here

- If the system is stuck in a local optimum, it can temporarily move to a worse timing plan and later find a better one.
- It works well for continuous real-time optimization problems.

Genetic Algorithms

Genetic Algorithms work with a population of solutions rather than one solution.

- Each solution is like a chromosome containing signal timings.

- Good solutions are selected and combined using crossover.
- Random mutation adds variation and helps exploration.

How it helps here

- It searches many possible timing plans at once.
- It is suitable for large-scale problems with many intersections.
- It can discover better global solutions than Hill Climbing.

Comparison

- Simulated Annealing is good for controlled exploration and dynamic adaptation.
- Genetic Algorithms are better for large, complex search spaces with many interacting variables.

V. Best approach and justification

The **best approach** for this smart city traffic optimization problem is **Genetic Algorithms**, possibly combined with real-time adaptive updates.

Why Genetic Algorithms are best

- They are highly scalable for hundreds of intersections.
- They explore multiple solutions simultaneously, which is useful for large search spaces.
- They are less likely to get stuck in local optima.
- They adapt well to complex and changing traffic patterns.

Why not only Hill Climbing

- Hill Climbing is simple and fast, but it is weak against local optima and dynamic changes.
- It is not ideal for a citywide system with many intersections and continuous updates.

Final recommendation

For **scalability and adaptability**, Genetic Algorithms are the most suitable choice. If the system needs faster online adjustment, Simulated Annealing can

also be used as a lightweight alternative or combined with Genetic Algorithms for hybrid optimization.

Short conclusion

This problem is a large-scale, dynamic optimization problem. Traditional search is inefficient, Hill Climbing is simple but easily trapped, and local optima create major challenges. Genetic Algorithms are the best fit because they can handle large search spaces, avoid local traps, and adapt better to changing traffic conditions.

3.)

i. Why online search is needed

An offline search agent plans using a complete model of the environment, but the Mars rover has only partial sensing and no complete map. It must choose actions based on what it knows now, then revise its plan as new terrain data arrives.

An online search agent is better because it:

- Makes decisions in real time.
- Learns from the environment while moving.
- Updates its route when it encounters obstacles or hazards.
- Works even when the full state space is unknown.

ii. Challenges of partial observability and dynamic environment

Partial observability

The rover can only sense a limited area around it, so it may not know what lies ahead. This creates uncertainty about craters, rocks, slopes, and safe paths.

Dynamic environment

Even if the terrain itself is mostly static, the rover's **knowledge** changes dynamically as it explores. New hazards may be detected, safe paths may turn out to be blocked, and the best route may change after each move.

Main difficulties

- The rover cannot plan a fully optimal route from the start.
- It may take a wrong step if the heuristic or local information is incomplete.
- It must balance exploration and safety.
- Delayed sensing can increase the risk of getting trapped or damaged.

iii. How the rover builds and updates its internal model

The rover maintains an **internal map or belief model** of the terrain. This model is initially incomplete and becomes more accurate as the rover gathers sensor data.

Process

- It starts with a small local map or unknown terrain representation.
- Sensors detect nearby objects, slopes, and hazards.
- Newly observed information is stored in memory.
- The rover updates the map and marks safe, risky, and blocked areas.
- It replans based on the updated internal model.

Example

If the rover first assumes a path is clear but later detects rocks, it updates that cell as blocked and chooses another path. This continuous revision is the core of online decision-making.

iV. Online search vs uninformed and informed search

Strategy	Main idea	Suitability here
Uninformed search	Explores without heuristic guidance	Not efficient because the rover has limited time and energy.
Informed search	Uses heuristics like estimated distance to goal	Better than uninformed search, but still assumes a known search space.

Strategy	Main idea	Suitability here
Online search	Makes decisions while learning the environment	Most suitable because the rover does not know the full map.

Comparison

- **Uninformed search** is too costly because it may explore many unnecessary states.
- **Informed search** can help when some map knowledge exists, but it still struggles when the environment is unknown.
- **Online search** is designed for uncertainty, so it fits the rover's mission much better.

V. Most suitable approach

The most suitable approach is an **online search agent with heuristic guidance**.

This gives the rover the ability to:

- Explore safely.
- Update its beliefs continuously.
- React to unexpected hazards.
- Make decisions with limited sensing and incomplete knowledge.

Why this is best

- **Uncertainty:** It handles unknown terrain naturally.
- **Adaptability:** It can replan as soon as new information appears.
- **Safety:** It avoids committing to a full path that may later prove dangerous.

Final justification

For a Mars rover, safety and adaptability are more important than finding one perfect offline route. Since the rover must operate in an unknown and partially observable environment, online search is the most practical and reliable strategy.

4.

1. Formulate as a CSP

Exam timetable generation can be modeled as a **Constraint Satisfaction**

Problem where the goal is to assign exam times, halls, and possibly invigilators to each course without violating any rules. The system must find a valid schedule that satisfies all constraints.

2. Variables, domains, and constraints

Variables

Each course/exam is a variable, such as X_1, X_2, X_3 , where each variable represents the scheduled slot for one exam.

Domains

The domain of each variable is the set of possible time slots and halls available for that exam.

Constraints

- No student should have two exams at the same time.
- A hall cannot be assigned to more than one exam in the same slot.
- Prerequisite-based subjects must be scheduled in the correct order.
- Faculty and special accommodation requirements must be respected.

These constraints ensure the timetable is valid and practical.

3. Backtracking search

Backtracking search assigns values to variables one by one. If a partial timetable violates a constraint, the algorithm goes back and tries a different assignment.

For example, if two exams for the same student are placed in the same slot, backtracking removes that choice and searches for another.

4. Constraint propagation and forward checking

Constraint propagation reduces the search space by removing impossible choices early. In **forward checking**, after assigning one exam, the system

checks remaining unassigned exams and removes time slots that would cause conflicts.

This improves efficiency because it prevents the algorithm from wasting time on invalid schedules.

5. Real-world challenges and best strategy

Challenges

- The problem becomes very large as courses and students increase.
- Many constraints interact with each other.
- Some schedules may be hard to find quickly.
- Changes in faculty or hall availability can require rescheduling.

Best strategy

A **backtracking search with constraint propagation and good heuristics** is the best scalable approach. Heuristics like choosing the most constrained exam first can reduce search time and make the system more efficient.

5

1. How Minimax models the problem

The **Minimax algorithm** models the game as a two-player decision tree where one player tries to maximize the score and the opponent tries to minimize it. In this scenario, the AI assumes the human player will make the best possible counter-move, so it chooses the move that gives the best guaranteed outcome.

At each level of the tree:

- The AI selects the move with the highest value.
- The opponent selects the move with the lowest value.

This helps the AI decide based on future consequences, not just the current move.

2. Computational challenges of large game trees

Real-time strategy games usually have a huge number of possible moves, so the game tree grows very quickly. This creates several problems:

- **High branching factor:** each state may lead to many possible moves.
- **Deep search cost:** exploring many levels takes too much time.
- **Strict time limits:** the AI must respond quickly during gameplay.
- **Complex evaluation:** exact terminal outcomes are often unavailable, so the AI must estimate state quality.

Because of these factors, full Minimax search becomes impractical in large games.

3. How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning is an optimization of Minimax that avoids exploring branches that cannot affect the final decision.

How it works

- **Alpha** stores the best value found so far for the maximizing player.
- **Beta** stores the best value found so far for the minimizing player.
- If a branch cannot produce a better result than the current alpha or beta bound, it is cut off.

Why it helps

- It reduces the number of nodes evaluated.
- It allows the AI to search deeper in the same amount of time.
- It produces the same final result as Minimax, but faster.

Detailed reasoning

Suppose the AI already found a move that gives a strong guaranteed score. If another branch cannot possibly beat that score, there is no need to continue searching it. This saves time without changing the decision quality.

4. Evaluation function design

Since the AI cannot search to the end of every game, it needs an **evaluation function** to estimate how good a game state is.

Possible factors

- Unit count and strength.
- Resource availability.

- Territory control.
- Army positioning and mobility.
- Health of important units.
- Threat level from opponent attacks.
- Progress toward victory objectives.

Justification

These factors matter because they reflect both current advantage and future potential. A good evaluation function should reward strong positions, economic stability, and tactical safety, while penalizing weak defense or wasted resources.

Example

A game state with more units, better map control, and higher resources should get a higher score than a state where the AI is exposed and low on resources.

5. Best strategy for balancing optimality and real-time performance

The best approach is to use **Minimax with Alpha-Beta pruning plus depth limiting and heuristic evaluation**.

Why this is the best balance

- Full Minimax is too slow for real-time play.
- Alpha-Beta pruning reduces unnecessary computation.
- Depth limiting ensures the AI responds within time constraints.
- The evaluation function lets the AI make strong decisions without searching to the end.

Final recommendation

For real-time strategy games, the AI should use **iterative deepening with Alpha-Beta pruning**. This gives a good balance of speed, adaptability, and decision quality, allowing the system to make usable moves even under strict time limits.

